

AR Castle Defense

Joc mòbil de defensa de torres en Realitat Augmentada: documentació tècnica i de disseny

1. Nom i descripció general de l'aplicació

AR Castle Defense és un joc de defensa de torres en Realitat Augmentada a escala d'habitació, desenvolupat amb Unity 6 i AR Foundation. Combina el seguiment òptic d'imatges amb la detecció de plans horitzontals en temps real per convertir qualsevol entorn físic pla en un camp de batalla medieval interactiu. Els jugadors ancoren una fortalesa 3D a l'espai real i despleguen canons defensius sobre les superfícies properes per derrotar les onades d'esquelets invasors.

Element	Detall
Nom de l'aplicació	AR Castle Defense
Plataforma objectiu	Android / iOS (AR mòbil)
Motor	Unity 6 (6000.4.9f1)
Framework d'AR	AR Foundation 6.4.3 (ARCore / ARKit)
Pipeline de renderitzat	URP
Repositori	AR-Foundation
Data de la documentació	Octubre de 2026

2. Objectiu de l'aplicació

L'objectiu principal és protegir el castell de les onades progressives d'esquelets invasors al llarg de 5 rondes de dificultat creixent, gestionant de manera tàctica el desplegament de canons defensius, limitat per recursos, sobre plans del món real.

2.1 Regles de joc i condicions de victòria

- **Defensar la fortalesa:** El castell comença amb 100 PV. Els esquelets infligeixen 10 de dany quan arriben a la porta del castell.
- **Condicció de victòria:** eliminar totes les onades enemigues de les 5 rondes sense que els PV del castell arribin a 0.
- **Condicció de derrota:** si els PV del castell arriben a 0, la fortalesa cau i s'activa una pantalla de derrota amb opcions de reinici.

2.2 Cicle de recursos i economia

- **Capital inicial:** 10 monedes d'or a l'inici de la sessió.
- **Cost de desplegament dels canons:** 5 monedes per canó desplegat tocant un pla.

- **Dividend per ronda superada:** +5 monedes en sobreviure a cada ronda.

3. Funcionalitats d'AR utilitzades

3.1 AR Tracked Image Manager (seguiment d'imatges)

El joc utilitza l'ARTrackedImageManager de Unity i una biblioteca de referència 2D precompilada (Castle_ImageTracking.jpeg).

- **Ancoratge físic fix:** el prefab 3D del Castell s'instancia i s'ancora estrictament a la posició i orientació del món físic del marcador.
- **Origen de coordenades:** la posició del marcador és el centre espacial cap al qual es calculen tots els vectors de navegació dels enemics i els anells d'aparició radials.
- **Reinici dinàmic del cicle de vida:** ARImageReset.cs permet als jugadors eliminar les entitats rastrejades i recalibrar sense reiniciar l'aplicació.

3.2 AR Plane Manager i Raycast Manager

El joc utilitza ARPlaneManager amb seguiment de polígons horitzontals i ARRaycastManager per a la col·locació espacial.

- **Comprensió de l'entorn:** els terres i les taules del món real s'escanegen en temps real, i es projecten malles visuals dels seus límits.
- **Raigs de pantalla a món:** s'intercepten els tocs a la pantalla i es llancen raigs contra els plans físics detectats per col·locar els canons defensius.
- **Comprovacions de validació:** la col·locació es valida segons les regles d'economia (CoinManager) i l'estabilitat de la superfície abans d'instanciar.

3.3 Interfície espacial d'AR i orientació cap a la càmera (billboarding)

L'AR mòbil requereix una retroalimentació dins del món que continuï essent llegible mentre el jugador es mou per l'habitació. El sistema integra Billboard.cs i el seguiment dinàmic de la càmera a SkeletonBehavior.LateUpdate(), que alinea les barres de vida 3D flotants de TextMeshPro perquè mirin sempre cap a Camera.main.transform, independentment de l'angle de visió.

4. Aspecte innovador

4.1 Arquitectura d'AR híbrida amb marcador i sense marcador

Els jocs d'AR tradicionals es basen només en la detecció de plans, que pateix una deriva espacial acumulada, o només en marcadors d'imatge, que limiten la interacció a una targeta petita. AR Castle Defense unifica ambdós paradigmes: la fortalesa central utilitza el seguiment per marcador per obtenir estabilitat espacial, mentre que les defenses col·locades pel jugador utilitzen raigs lliures sobre superfícies a escala d'habitació.

4.2 Balística amb línia de visió en AR a escala d'habitació

A diferència dels jocs mòbils de defensa de torres convencionals, on les defenses disparen a través de la geometria del nivell, els canons executen raigs de física en temps real (HasObstacleInBetween()) contra

obstacleLayer). Els canons identifiquen l'enemic visible més proper, giren cap a ell amb quaternions suaus i disparen projectils balístics amb col·lisió espacial real.

4.3 Economia de recursos lligada a la geometria física

La mecànica de col·locació lliga l'economia del jugador (10 monedes inicials, 5 monedes per canó) amb la topologia física real de l'habitació. Els jugadors no poden omplir el camp de torres; han de caminar per l'habitació per inspeccionar angles, punts morts de la línia de visió abans de gastar or en una posició de canó.

4.4 Escalat autònom de rondes i resiliència del seguiment

La progressió de les onades augmenta un 20% el nombre d'enemics i redueix un 20% els intervals d'aparició a cada ronda, i introdueix esquelets d'espasa blindats amb més durabilitat. Un mecanisme de recuperació integrat (ARImageReset) elimina les entitats rastrejades i restaura de manera segura l'estat del joc si el seguiment òptic es degrada a l'entorn.

5. Resum del procés de desenvolupament

El desenvolupament va seguir un enfocament iteratiu, basat en fites, amb control de versions Git i compilacions mòbils URP de Unity 6. El projecte va avançar per 5 fites principals, des d'un prototip base fins a una versió final completa.

5.1 Fites

Fita	Implementacions principals	Detalls tècnics i scripts	Lliurables
Fase 1: configuració i base	Configuració del projecte, pipeline URP, paquets XR, configuració de Git	AR Foundation 6.4.3, ARCore / ARKit, XR Interaction Toolkit, .gitignore per a LFS	Repositori net amb el feed de càmera d'AR mòbil verificat
Builds 1.0 i 2.0: ancoratge espacial	Transició del prototip de toc sobre plans al seguiment òptic d'imatges	ReferenceImageLibrary New, targeta marcador, raigs sobre plans tàctils per als canons	Prefab del Castell ancorat i lògica inicial d'aparició de canons amb un toc
Build 3.0: generadors i IA	Generador radial d'enemics i IA de navegació direccional	EnemySpawner.cs (distribució radial en cercle), SkeletonBehavior.cs (vector frontal cap al castell)	Moviment automàtic dels esquelets cap a l'àncora de la fortalesa
Build 4.0: combat i mecàniques	Adquisició d'objectius pels canons, balística, sistemes de vida i	CannonBehavior.cs (rotació, comprovació d'obstacles), Bullet.cs,	Cicle de combat funcional, aplicació de

Fita	Implementacions principals	Detalls tècnics i scripts	Lliurables
	estats de victòria i derrota	CastleBehavior.cs (vida i lògica de rondes)	dany i progressió de rondes
Build 5.0: economia i polit de la IU	Gestor d'economia, límit de canons, variants d'enemics, flux complet d'IU	CoinManager.cs, CannonSpawnValidator.cs, GameUIManager.cs, barres de vida billboard, esquelets d'espasa	Versió final completa amb menú principal, HUD, final de partida i reinici

5.2 Visió general de l'arquitectura: scripts principals

Script	Responsabilitat principal	Mètodes / API clau
CastleBehavior.cs	Fa el seguiment de la salut de la fortalesa, gestiona les 5 rondes d'onades, escala la dificultat i atorga recompenses en monedes	TakeDamage(), UpdateHealth(), End(), Reset()
CannonBehavior.cs	Escaneja els enemics propers, avalua la línia de visió amb obstacles, gira la torreta amb suavitat i dispara projectils	HasObstacleInBetween(), Shoot(), OnTriggerEnter()
CoinManager.cs	Controlador d'economia central (singleton): porta el saldo de monedes, valida les col·locacions i aplica el límit de 3 canons	CanSpawnCannon(), RegisterCannonSpawned()
EnemySpawner.cs	Genera enemics de manera radial al voltant del castell; tria entre esquelets estàndard i esquelets d'espasa blindats	SpawnSkeletonAroundCastle(), ChooseEnemy()
SkeletonBehavior.cs	Es dirigeix cap al castell, causa dany en col·lisionar, gestiona els PV i el text de vida billboard orientat a la càmera	TakeDamage(), Die(), LateUpdate() (Billboard)
GameUIManager.cs	Singleton que gestiona tot el cicle de vida de la IU: menú principal, finestra d'instruccions,	UpdateWaveInfo(), ShowGameOver(), Reset()

Script	Responsabilitat principal	Mètodes / API clau
	HUD en partida, final de partida i victòria	

6. Problemes trobats i solucions aplicades

6.1 Desplaçament del pivot 3D del canó i rotació errònia sobre el terra

- **Síntoma:** en apuntar als esquelets que s'acostaven, la malla del canó s'inclinava respecte a la seva base, travessava el terra o girava al voltant d'un eix de pivot excèntric i descentrat en lloc de rotar sobre si mateixa.
- **Causa arrel:** l'asset 3D importat tenia el punt de pivot en una cantonada arbitrària i no al centre de la base. L'aplicació directa d'un LookRotation 3D sense restriccions provocava canvis involuntaris de capcineig i balanceig en eixos no restringits.
- **Solució:** es va reestructurar la jerarquia del prefab imbricant la malla visual dins d'un pare buit i centrat. A CannonBehavior.cs es va aïllar la rotació de guinyada al voltant de l'eix vertical (Vector3.up), es va descartar la inclinació vertical i es va suavitzar el moviment amb Quaternion.RotateTowards(transform.rotation, targetRotation, rotSpeed * Time.deltaTime).

6.2 Oclusió de projectils i detecció de línia de visió sense obstacles

- **Síntoma:** els canons apuntaven amb freqüència a esquelets situats just darrere del castell central o darrere d'obstacles de l'escena, i disparaven les bales directament contra la geometria amiga, malgastant munició.
- **Causa arrel:** la selecció d'objectius es basava només en la distància euclidiana mínima (Vector3.Distance) dins d'un col·lisionador de tipus trigger superposat, ignorant les barreres físiques intermèdies.
- **Solució:** es va desenvolupar el mètode HasObstacleInBetween(Vector3 targetPosition). Abans de fixar un objectiu, el canó llança un raig de física des del seu firePoint cap a l'enemic sobre una obstacleLayer dedicada. Si es detecta un obstacle, el candidat es descarta en favor d'objectius sense obstruccions.

6.3 Toc ràpid repetitiu i instanciació il·limitada de canons

- **Síntoma:** els jugadors podien tocar ràpidament els plans d'AR detectats per generar desenes de canons, cosa que degradava la taxa de fotogrames del mòbil, esgotava la memòria i trivialitzava el repte.
- **Causa arrel:** els oients de tocs sobre els plans d'AR invocaven directament la instanciació, sense comprovar si es podia pagar el recurs ni els límits d'entitats.
- **Solució:** es van implementar un singleton d'economia centralitzat (CoinManager) i un component de seguretat (CannonSpawnValidator). En activar-se, CanSpawnCannon() verifica que el nombre actual de canons sigui inferior al límit màxim (3) i que el saldo sigui d'almenys 5 monedes. Les instanciacions que no compleixen els requisits s'avorten immediatament, i les ranures es reemborsen o es recuperen automàticament en reiniciar l'escena.

6.4 Pèrdua de seguiment d'AR, tremolor del marcador i aparicions duplicades del castell

- **Síntoma:** amb canvis d'il·luminació a l'habitació o girs ràpids del dispositiu, el seguiment d'imatges s'interrompia de manera intermitent, i feia que el castell derivés, desaparegués o es duplicués en tornar-lo a detectar.
- **Causa arrel:** l'ARTrackedImageManager de Unity conservava dades de seguiment obsoletes i no oferia cap mecanisme integrat d'autoneteja per als elements rastrejats perduts.
- **Solució:** es va crear ARImageReset.cs i es va connectar al HUD de la partida. Destruïx de manera segura les instàncies rastrejades existents, reinicia el comptador de torretes actives, desactiva i torna a activar el component ARTrackedImageManager per netejar els buffers dels sensors, i reancora el món del joc.

6.5 Llegibilitat del text 3D dins del món des d'angles de visió d'AR dinàmics

- **Síntoma:** els valors de vida dels enemics mostrats sobre el cap dels esquelets apareixien del revés, distorsionats o il·legibles quan l'usuari caminava pel perímetre d'AR.
- **Causa arrel:** una rotació fixa en l'espai del món feia que el text només mirés cap a l'eix Z del món en lloc d'ajustar-se al punt de vista canviant del telèfon.
- **Solució:** es va implementar Billboard.cs, amb l'orientació billboard en temps real també integrada a SkeletonBehavior.LateUpdate():
`Quaternion.LookRotation(healthText.transform.position - mainCameraTransform.position)`. Això alinea contínuament els indicadors flotants de manera perpendicular a la càmera del dispositiu.

7. Contribució de cada membre de l'equip

Aquesta secció documenta les tasques concretes, els scripts, els elements d'interfície, els procediments de prova i la documentació lliurats per cada membre.

7.1 Pau Gutsens: responsable d'UI / UX, integració de sistemes

- **Àmbits de treball:** disseny i flux de la IU / HUD; scripts de joc en C#; arquitectura i comentaris de codi; localització i polit de textos; proves en dispositiu d'AR.
- **Scripts / components assignats:** GameManager.cs, activadors d'esdeveniments d'IU, integració de l'ensamblatge.
- **Tasques concretes realitzades:** va dissenyar i implementar des de zero el flux complet d'IU (menú principal, finestra d'instruccions, HUD en partida amb barra de PV del castell, comptadors de rondes, enemics restants i bàners de final de partida / victòria). Es va encarregar dels comentaris de codi i les traduccions, va integrar els enllaços d'IU a tots els scripts de joc i va realitzar proves en dispositius mòbils.

7.2 Martí Mira: sistemes d'economia, arquitecte del projecte

- **Àmbits de treball:** scripts de joc en C#; economia i equilibri del joc; integració de l'escena mestra; configuració del projecte; documentació tècnica; lògica de rotació de les torretes.
- **Scripts / components assignats:** CoinManager.cs, configuració de TowerDefense.unity, README.md, Project Settings.
- **Tasques concretes realitzades:** va inicialitzar la configuració del projecte Unity i els ajustos del pipeline. Va dissenyar i construir el sistema d'economia CoinManager complet (saldo de

monedes, límit màxim de 3 canons, incentius de recompensa per ronda). Va treballar en els algorismes de rotació dels canons, va afinar l'escena mestra TowerDefense i va redactar la documentació del projecte.

7.3 Arnau Balasch: AR Foundation, generador i mecàniques d'enemics

- **Àmbits de treball:** seguiment i ancoratge d'imatges d'AR; IA d'enemics i aparició radial; prefabs 3D i alineació de pivots; sistema de tret dels canons; Git ignore i control de versions; variants d'enemics (esquelet d'espasa).
- **Scripts / components assignats:** EnemySpawner.cs, SkeletonBehavior.cs, CannonSpawnValidator.cs, biblioteca d'imatges de referència.
- **Tasques concretes realitzades:** va construir els prototips inicials de les builds 1.0 i 2.0 integrant el seguiment d'imatges d'AR. Va dissenyar l'EnemySpawner radial i el SkeletonBehavior amb barres de vida billboard. Va corregir els problemes d'alineació del pivot 3D del canó, va redactar la lògica de llançament de projectils i va integrar nous arquetips d'enemics (esquelets d'espasa amb escalat de vida).

7.4 Marc Blánquez: IA de combat, balística i condicions de victòria

- **Àmbits de treball:** IA de combat dels canons; raigs de línia de visió; condicions de victòria i derrota; escalat de la progressió d'onades; sistemes de dany i col·lisions; proves de joc en AR.
- **Scripts / components assignats:** CannonBehavior.cs, Bullet.cs, CastleBehavior.cs (integració de victòria / derrota).
- **Tasques concretes realitzades:** va implementar la IA d'objectius de combat dels canons i el sistema de raigs d'obstacles de línia de visió per evitar disparar a través de barreres. Va desenvolupar la lògica de rondes de CastleBehavior, els multiplicadors de dificultat progressiva i l'avaluació dels estats de victòria / derrota. Va realitzar proves de col·lisió entre els projectils i els col·lisionadors dels enemics.